

Deployment ML-моделей: от прототипа к продакшену

Введение

Обучили модель, хотим сохранить и использовать.

- Деплой — это не `model.save()`
- Модель в Jupyter $\neq$ модель в реальном продукте

Jupyter vs Production

- Входные данные

`jupyter` чистый CSV, фиксированный набор

`prod` произвольные запросы: пропуски, выбросы, новые категории

- Нагрузка

`jupyter` одно предсказание вручную

`prod` до миллионов запросов/сек

- Время ответа

`jupyter` минуты (обучение), доли секунды (инференс)

`prod` жёсткие ограничения

- Инфраструктура

`jupyter` одна машина, все ресурсы — наши

`prod` множество серверов, конкуренция за CPU/GPU, сеть

- Обновление

`jupyter` обновление и перезапуск ячейки

`prod` Canary-деплой, A/B-тесты, возможность отката

- Ошибки

`jupyter` видим traceback, исправляем

`prod` ошибка затрагивает реальных пользователей

Примеры «сбоев» при наивном деплое

- ❶ **Несоответствие данных:** Модель ожидает колонку типа `int`, а приходит `"NaN"` строка. → падение всего сервиса.
- ❷ **Проблемы параллелизма:** Модель предсказывает за 300 мс, но при 500 одновременных запросах время ответа вырастает до 5 секунд.
- ❸ **Незафиксированная среда:** Переобучили модель с новой версией библиотеки — `precision` упал вдвое без предупреждения.
- ❹ **Отсутствие мониторинга:** Модель деградирует из-за дрейфа данных, а узнаём мы об этом от жалоб пользователей.

MLOps vs DevOps

Критерий	DevOps	MLOps
Артефакт	Код (детерминированный)	Модель + код + данные (вероятностные)
Входные данные	Фиксированный формат запросов (API)	Распределение может дрейфовать
Ключевые тесты	Unit, integration	+ Проверка на дрейф, стабильность предсказаний
Основные метрики	Uptime, latency, error rate	+ Accuracy, precision, drift-метрики
Стратегия обновления	Blue-green, canary, rollback	+ Периодическое переобучение

Метрики продакшен-модели

В проде мы оптимизируем не только качество предсказаний, но и экономику сервиса

- **Latency (задержка)** — время от запроса до ответа. Перцентили p_{95} , p_{99} , а не среднее.
- **Throughput (пропускная способность)** — число запросов в секунду (RPS), которые система держит без деградации p_{99} .
- **Cost (стоимость инференса)**. cost per prediction $\frac{\text{GPU-часы} \times \text{цена}}{\text{число предсказаний}}$
- **Drift (дрейф)** — распределение входов $P(X)$ меняется (новый регион, сезон). Модель «стареет», даже если код не менялся.

Ключевые вопросы

❶ Где запускать модель?

Устройство и окружение: Cloud, Edge, On-premise, Serverless

❷ Как быстро отвечать?

Режим инференса: real-time, batch, streaming

❸ Обновлять ли модель?

Статическая модель vs дообучение, A/B-тесты, canary

❹ Как упаковать и сервировать?

Форматы (ONNX, SavedModel), фреймворки (Triton, BentoML), Docker

❺ Что делать, когда что-то пошло не так?

Мониторинг, дрейф, масштабирование, отказоустойчивость

Где запускать модель?

Платформа	Плюсы	Минусы	Инструменты
Cloud (VM/K8s)	Масштабируемость, managed-сервисы	Стоимость, поддержка сети	AWS SageMaker, Vertex AI, Kubernetes
On-premise	Контроль данных, низкая задержка	CAPEX, обслуживание	Собственное железо, Kubernetes
Edge / Mobile	Оффлайн-работа, минимальная задержка	Ограниченные ресурсы	TFLite, ONNX Runtime, TF.js
Serverless	Оплата за вызов, автоскейлинг	Холодный старт, лимиты памяти	AWS Lambda, Cloud Run

Модель на Edge

Ограниченные ресурсы

- Мобильные устройства (CPU, GPU)
- Микроконтроллеры (MCU)
- FPGA

Сжатие и оптимизация

- **Квантование:** снижение точности весов ($FP32 \rightarrow INT8$), ускорение на CPU/GPU.
- **Прунинг:** удаление малоинформативных весов.
- **Дистилляция:** обучение лёгкой «студенческой» модели на предсказаниях «учителя».
- **TensorRT / OpenVINO:** оптимизация графа вычислений под конкретное железо.

Режим инференса

- **Real-time** — модель обрабатывает каждый запрос индивидуально и немедленно, по мере поступления.
- **Batch** — модель обрабатывает данные большими порциями по расписанию или по накоплению.
- **Streaming** — непрерывная обработка событий в реальном времени с сохранением состояния.

Гибридные архитектуры

- **Лямбда-архитектура**: batch-слой (полный пересчёт ночью) + real-time слой (поточковая корректировка).
- **Micro-batching**: сервер накапливает запросы за 100–500 мс и прогоняет пачку через GPU. Резко повышает пропускную способность.
- **Pre-compute + Cache**: для рекомендательных систем — популярные товары/профили кэшируются и обновляются раз в час.

Жизненный цикл модели

- Статическая — обучили один раз и задеплоили, нет пайплайна переобучения, быстрое устаревание.
- Периодическая — переобучение по расписанию или триггеру, автоматический пайплайн (MLflow, Kubeflow), баланс свежести и стабильности
- Онлайн — непрерывное обновление, потоковое обучение (River, Vowpal Wabbit), риск catastrophic forgetting (забывание) и data poisoning (отравление данных)

Чеклист: дообучать ли?

- Есть (измерен) ли дрейф данных?
- Достаточно ли свежих размеченных данных?
- Допускает ли бизнес задержку на переобучение?
- Настроена ли инфраструктура A/B-тестов?

Безопасное обновление: Canary и A/B

- **Canary деплой:** 2–5% трафика на новую модель, если метрики не упали — расширяем.
- **A/B-тест:** параллельно работают две версии, сравниваются бизнес-показатели (конверсия, доход).
- **Shadow mode:** новая модель работает параллельно, но её предсказания не идут в дело — только логируются. Позволяет безопасно накопить статистику.

Упаковка и инструменты сервинга

Формат сериализации определяет **скорость загрузки, безопасность и зависимость от окружения**.

- `pickle` / `joblib` — только Python, зависимость от версий, небезопасно.
- **ONNX** — открытый формат, экспорт из PyTorch/TF/sklearn, быстрый рантайм (ONNX Runtime).
- **TorchScript** — JIT-компиляция PyTorch, без привязки к Python.
- **SavedModel** — нативный формат TensorFlow для TF Serving.

Форматы сериализации для продакшена

Библиотека	Нативный формат	Кроссплатформенные форматы	Инструменты конвертации
sklearn	pickle, joblib	ONNX, PMML, Treelite	skl2onnx, sklearn2pmml
CatBoost	.cbm, .cpp, .mlmodel	ONNX, PMML, Treelite	встроенный save_model (ONNX), treelite
PyTorch	.pth, TorchScript (.pt)	ONNX, TorchScript	torch.onnx.export, torch.jit.script
TensorFlow	SavedModel, .h5	ONNX, TF Lite	tf2onnx, TFLiteConverter

Сервинг-фреймворки

Тип	Инструменты	Особенности
Универсальные	FastAPI, BentoML, MLflow Models	Простота, гибкость, управление версиями
Deep Learning	Triton, TorchServe, TF Serving	GPU, dynamic batching, высокая производительность
LLM	vLLM, TGI, OpenLLM	Кеширование KV, PagedAttention, streaming токенов

Классические ML

CPU-инференс, легковесный API.

Пример: XGBoost через FastAPI.

Глубокое обучение

GPU, ONNX/TensorRT.

Пример: Triton + ResNet.

Типичные антипаттерны деплоя

❶ «Работает на моей машине»

Нет контейнеризации (Docker). Воспроизводимость нулевая.

❷ Нет мониторинга

Дрейф, ошибки, задержки — всё обнаруживается через жалобы клиентов.

❸ Жёсткая связка код-модель-данные

Нельзя обновить модель отдельно от API. Каждый релиз — стресс.

❹ Отсутствие fallback-стратегий

Модель упала → сервис лежит. Нет дефолтного ответа, нет кэша.

❺ Игнорирование валидации входа

Непроверенные данные ломают модель или портят бизнес-показатели.

Мониторинг в проде

Системный слой

- Latency (p50, p95, p99)
- Throughput (RPS)
- Error rate (4xx/5xx)
- CPU/GPU/memory

Модельный слой

- Data drift
- Concept drift
- Prediction drift

Бизнес-слой

Конверсия, выручка, отток — настоящая ценность модели.

Банк выдает кредиты. Модель обучена в 2019 году.

- 1 Data Drift — изменяется распределение входных признаков $P(X)$, при этом $P(y|X)$ стабильна (распределение доходов заёмщиков сдвинулось влево, больше низких доходов).
- 2 Concept drift — изменяется $P(y|X)$ (низкий доход теперь сильнее коррелирует с дефолтом, чем раньше)
- 3 Prediction drift — изменяется $P(\hat{y})$. Симптом, не причина. (модель стала чаще предсказывать «дефолт»)

Data Drift: метрики и инструменты

Отсутствие дрейф-мониторинга — главная причина «тихой смерти» моделей.

- **PSI (Population Stability Index)** — оценивает разницу между распределением признака в train и в production.

$$PSI = \sum_i (P_{prod}(i) - P_{train}(i)) \cdot \ln \frac{P_{prod}(i)}{P_{train}(i)}$$

- **KS-test** — статистический тест на различие распределений.
- Инструменты: Evidently AI, WhyLabs, Alibi Detect.

Инструменты мониторинга

Назначение	Инструменты	Что мониторит
Системный	Prometheus + Grafana	Latency, CPU, ошибки
Дрейф данных и моделей	Evidently AI, Arize	Data drift, concept drift
Логирование	ELK, Loki	Запросы, ответы, ошибки

Масштабирование и отказоустойчивость

- **Горизонтальное масштабирование:** несколько реплик модели за балансировщиком.
- **Очереди** (Redis, RabbitMQ, Kafka) — буферизация запросов при пиках.
- **Fallback:** дефолтный ответ или результат из кэша, если модель не отвечает.
- **Rollback:** автоматический откат к предыдущей версии модели по триггеру (Evidently + CI/CD).
- **Health checks** (liveness/readiness probes) — Kubernetes рестартует зависшие поды.